

INTRODUCCIÓN A LA PROGRAMACIÓN ORIENTADA A OBJETOS

Genericidad

Dr. Luciano H. Tamargo
http://cs.uns.edu.ar/~lt
Depto. de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur, Bahía Blanca

GENERICIDAD

- Los datos de aplicaciones muy diferentes con frecuencia pueden representarse con estructuras a las que se accede con operaciones que no dependen del tipo de las componentes.

- Una **clase genérica** encapsula a una estructura cuyo comportamiento es independiente del tipo de las componentes.

- La **genericidad** favorece la **reusabilidad**.

GENERICIDAD

- La clase **Inventario** encapsula una **colección** de componentes de tipo **Articulo**, representada a través de un arreglo parcialmente ocupado.
- El inventario respeta el orden de inserción.
- Las componentes están comprimidas, de modo que si se almacenan `cantArticulos`, ocupan las primeras `cantArticulos` posiciones del arreglo.



GENERICIDAD

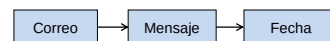
Inventario	Articulo
<pre> t [] Articulo cantArticulos: entero <<Constructores>> Inventario(max: entero) <<Comandos>> insertar(c: Articulo) eliminar(c: Articulo) depreciarRubro(r: entero, p: real) <<Consultas>> cantArticulos(): entero estaLlena(): boolean pertenece(c: Articulo): boolean unAnio(a: entero): Inventario </pre>	<pre> codigo: entero rubro: entero valor: real anio: entero <<Constructor>> Articulo(c: entero, r: entero, v: entero, a: entero) <<Comandos>> depreciar(p: real) <<Consultas>> equals(a: Articulo): boolean mayor(a: Articulo): boolean </pre>

GENERICIDAD

- Inventario(max: entero)** crea una colección con capacidad para mantener `max` artículos.
- insertar(c: Articulo)** asigna el artículo `c` a la primera posición libre e incrementa `cantArticulos`. Requiere `c` ligado y `estaLlena() == false`.
- eliminar(c: Articulo)** busca un artículo equivalente a `c`, si existe, lo elimina arrastrando los que le siguen una posición y decrementando `cantArticulos`.
- depreciarRubro(r: entero, p: real)** recorre exhaustivamente la colección y deprecia cada artículo del rubro `r` en el porcentaje `p`.
- cantArticulos(): entero** retorna la cantidad de artículos almacenados en la colección.
- estaLlena(): boolean** retorna `true` si la cantidad de artículos es igual al tamaño de la colección.
- pertenece(c: Articulo): boolean** retorna `true` si la colección contiene un elemento equivalente a `c`.
- unAnio(a: entero): Inventario** retorna una colección con los artículos que corresponden al año `a` en la colección que recibe el mensaje.

GENERICIDAD

- La clase **Correo** encapsula una **colección** de componentes de tipo **Mensaje**, representada a través de un arreglo parcialmente ocupado.
- El correo respeta el orden de inserción.
- Las componentes están comprimidas, de modo que si se almacenan `cantMensajes`, ocupan las primeras `cantMensajes` posiciones del arreglo.



GENERICIDAD

Correo	Mensaje
<code>t [] Mensaje</code>	<code>contacto: Contacto</code>
<code>cantMensajes: entero</code>	<code>fecha: Fecha</code>
<code><<Constructores>></code>	<code>hora: Hora</code>
<code>Correo(max: entero)</code>	<code>asunto: String</code>
<code><<Comandos>></code>	<code>contenido: String</code>
<code>insertar(m: Mensaje)</code>	<code><<Constructor>></code>
<code>eliminar(m: Mensaje)</code>	<code>...</code>
<code><<Consultas>></code>	<code><<Comandos>></code>
<code>cantMensajes(): entero</code>	<code>...</code>
<code>estaLlena(): boolean</code>	<code><<Consultas>></code>
<code>pertenece(m: Mensaje): boolean</code>	<code>equals(m: Mensaje): boolean</code>
<code>filtroAsunto(a: String): Correo</code>	<code>mayor(m: Mensaje): boolean</code>

GENERICIDAD

- Correo(max: entero)** crea una colección con capacidad para mantener *max* mensajes.
- insertar(c: Mensaje)** asigna el mensaje a la primera posición libre e incrementa *cantMensajes*. Requiere *c* ligado y *estaLlena()* == false.
- eliminar(c: Mensaje)** busca un Mensaje equivalente a *c*, si existe, lo elimina arrastrando los que le siguen una posición y decrementando *cantMensajes*.
- cantMensajes(): entero** retorna la cantidad de *Mensajes* almacenados en la colección.
- estaLlena(): boolean** retorna *true* si la cantidad de *Mensajes* es igual al tamaño de la colección.
- pertenece(c: Mensaje): boolean** retorna *true* si la colección contiene un elemento equivalente a *c*.
- filtroAsunto(a: String): Correo** retorna una colección con los mensajes que corresponden al asunto *a* en la colección que recibe el mensaje.

GENERICIDAD

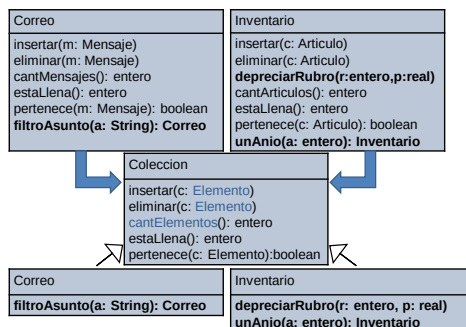
Correo	Inventario
<code>t [] Mensaje</code>	<code>t [] Artículo</code>
<code>cantMensajes: entero</code>	<code>cantArticulos: entero</code>
<code><<Constructores>></code>	<code><<Constructores>></code>
<code>Correo(max: entero)</code>	<code>Inventario(max: entero)</code>
<code><<Comandos>></code>	<code><<Comandos>></code>
<code>insertar(m: Mensaje)</code>	<code>insertar(c: Artículo)</code>
<code>eliminar(m: Mensaje)</code>	<code>eliminar(c: Artículo)</code>
<code><<Consultas>></code>	<code>depreciarRubro(r: entero, p: real)</code>
<code>cantMensajes(): entero</code>	<code><<Consultas>></code>
<code>estaLlena(): boolean</code>	<code>cantArticulos(): entero</code>
<code>pertenece(m: Mensaje): boolean</code>	<code>estaLlena(): boolean</code>
<code>filtroAsunto(a: String): Correo</code>	<code>pertenece(c: Artículo): boolean</code>
	<code>unAnio(a: entero): Inventario</code>

GENERICIDAD

Correo	Inventario
<code>t [] Mensaje</code>	<code>t [] Artículo</code>
<code>cantMensajes: entero</code>	<code>cantArticulos: entero</code>
<code><<Constructores>></code>	<code><<Constructores>></code>
<code>Correo(max: entero)</code>	<code>Inventario(max: entero)</code>

```

class Coleccion{
    //atributos de instancia
    protected Elemento [] t;
    protected int cant;
    //Constructor
    public Coleccion(int max){
        t = new Elemento [max];
        cantElementos = 0;
    }
}
    
```



GENERICIDAD

- Colección(max: entero)** crea una colección con capacidad para mantener *max* componentes.
- insertar(c: Elemento)** asigna el elemento *c* a la primera posición libre e incrementa *cantElementos*. Requiere *c* ligado y *estaLlena()* == false.
- eliminar(c: Elemento)** busca un *Elemento* equivalente a *c*, si existe, lo elimina arrastrando los que le siguen una posición y decrementando *cantElementos*.
- cantElementos(): entero** retorna la cantidad de *Elementos* almacenados en la colección.
- estaLlena(): entero** retorna *true* si la cantidad de *Elementos* es igual al tamaño de la colección.
- pertenece(c: Elemento): boolean** retorna *true* si la colección contiene un elemento equivalente a *c*.

GENERICIDAD

```
public void insertar(Mensaje m) {
    /*Inserta un mensaje en la primera posición libre. Requiere que la
    colección no esté llena y m no nulo*/
    t[cantMensajes] = m; cantMensajes++;
}

public void insertar(Articulo a) {
    /*Inserta un articulo en la primera posición libre. Requiere que la
    colección no esté llena y a no nulo*/
    t[cantArticulos] = a; cantArticulos++;
}

public void insertar(Elemento elem) {
    /*Inserta un elemento en la primera posición libre. Requiere que la
    colección no esté llena y elem no nulo*/
    t[cantElementos] = elem; cantElementos++;
}
```

GENERICIDAD

```
public int cantMensajes () {
    return cantMensajes;
}

public int cantArticulos() {
    return cantArticulos;
}

public int cantElementos() {
    return cantElementos;
}
```

- La definición de este método genérico solo implica un cambio de nombre.

GENERICIDAD

```
public void eliminar(Mensaje m){
    /*Busca un mensaje equivalente a m en la colección, si lo
    encuentra arrastra los mensajes que siguen una posición.
    Requiere m ligado*/
    boolean esta = false; int i= 0;
    while (!esta && i < cantMensajes())
        if (t[i].equals(m))
            esta = true;
        else
            i++;
    if (esta) {
        cantMensajes--;
        arrastrar(i);
    }
}
```

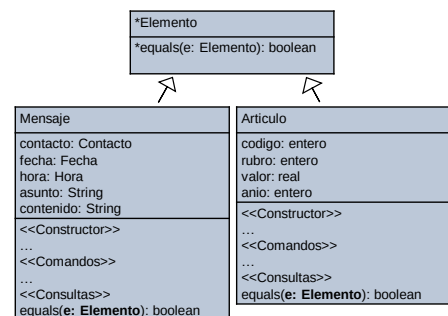
GENERICIDAD

```
public void eliminar(Articulo a){
    /*Busca un articulo equivalente a a en la colección, si lo
    encuentra arrastra los articulos que siguen una posición.
    Requiere a ligado*/
    boolean esta = false; int i= 0;
    while (!esta && i < cantArticulos())
        if (t[i].equals(a))
            esta = true;
        else
            i++;
    if (esta) {
        cantArticulos--;
        arrastrar(i);
    }
}
```

GENERICIDAD

```
public void eliminar(Elemento elem){
    /*Busca un elemento equivalente a elem en la colección, si
    lo encuentra arrastra los elementos que siguen una posición.
    Requiere elem ligado*/
    boolean esta = false; int i= 0;
    while (!esta && i < cantElementos())
        if (t[i].equals(elem))
            esta = true;
        else
            i++;
    if (esta) {
        cantElementos--;
        arrastrar(i);
    }
}
```

La ligadura dinámica vincula el mensaje `equals` con el método que corresponda, de acuerdo al tipo dinámico de la variable polimórfica `t[i]`.



GENERICIDAD

```
abstract class Elemento {
    abstract public boolean equals(Elemento e);
}
```

```
01100
10011
10110
01110
01100
10011
10110
01110
10011
1 11
0 0
1
```

19

GENERICIDAD

```
abstract class Elemento {
    abstract public boolean equals(Elemento e);
}
```

```
class Artículo extends Elemento {
    //Atributos de instancia
    //Constructor
    //Comandos
    //Consultas
    public boolean equals(Elemento e){
        ...
    }
}
```

```
0
1
0
0
0
1
0
0
0
```

20

GENERICIDAD

```
class Mensajeria{
    private Correo cor;
    public void gestioCorreo{

        cor = new Correo (n);
        Mensaje m = new Mensaje (...);

        cor.insertar(m);
    }
}
```

En la clase **Mensajeria** el objeto ligado a la variable **correo** siempre recibe el mensaje **insertar** con un parámetro de clase **Mensaje**.

```
0
0
0
0
0
0
0
0
0
```

21

- Un objeto de clase **Correo** puede recibir cualquiera de los mensajes que corresponden a servicios provistos por **Coleccion**.

GENERICIDAD

- Las componentes de una instancia de **Coleccion** tienen tipo estático **Elemento**, lo cual asegura que está definido el método **equals**.

```

Coleccion
...
insertar(c:Elemento)
eliminar(c:Elemento)
cantElementos():entero
estaLlena():entero
pertenece(c:Elemento):boolean
    
```

```
01100
10011
10110
01110
01100
10111
10110
01110
```

22

```

Correo
...
filtroAsunto(a: String): Correo
    
```

```

Inventario
...
depreciarRubro(r: entero,p: real)
unAnio(a: entero): Inventario
    
```

GENERICIDAD

```
class Correo extends Coleccion {
    public Correo(int max){
        super(max);
    }
    public Correo FiltroAsunto(String a){
        /*Retorna una colección con los mensajes que corresponden al
        asunto a en la colección que recibe el mensaje.*/
        Correo cor = new Correo(cantElementos());
        for (int i=0; i<cantElementos();i++){
            Mensaje sms = (Mensaje) t[i];
            if (sms.obtenerAsunto().equals(a))
                cor.insertar(t[i]);
        }
        return cor;
    }
}
```

Podemos asegurar que el tipo dinámico de **t[i]** es **Mensaje**.

```
0
0
0
0
0
0
0
0
0
```

23

GENERICIDAD

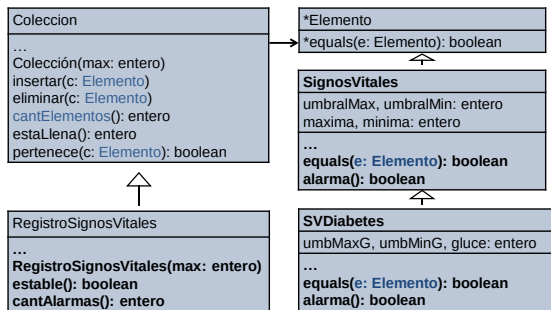
```
class Mensajeria{
    private Correo cor;
    public void gestioCorreo{
        cor = new Correo (n);
        Mensaje m = new Mensaje (...);
        cor.insertar(m);

        String unAsunto = ...;
        Correo otro = cor.filtroAsunto(unAsunto);
    }
}
```

```
0
1
0
0
0
0
0
1
0
```

24

- Un objeto de clase **Correo** puede recibir cualquiera de los mensajes que corresponden a servicios provistos por su propia clase.



25

```

class ControlPaciente{
    private RegistroSignosVitales rs;
    public void procedimientoControl{

        rs = new RegistroSignosVitales (40);
        SignosVitales mañana,tarde;
        mañana = new SVDiabetes(...);
        tarde = new SVDiabetes(...);

        ...
        rs.insertar(mañana);
        rs.insertar(tarde);
        ...

        if (rs.cantAlarmas>2) ...
    }
}
    
```

En la clase **ControlPaciente** el objeto ligado a la variable **rs** siempre recibe el mensaje **insertar** con un parámetro que pertenece a la clase **SignosVitales**.

- La clase **ControlPaciente** es responsable de garantizar que se cumplen los requerimientos asumidos por la clase **SignosVitales**.

26

```

class RegistroSignosVitales extends Coleccion{
    public RegistroSignosVitales(int max){
        super(max);
    }
    public boolean cantAlarmas(){
        /*Computa la cantidad de registros que indican alarma en
        la temperatura, la presión o la glucemia en el caso de
        pacientes diabéticos a los que se les realiza esta
        medición*/
        int cant=0;
        for (int i = 0; i < cantElementos(); i++){
            SignosVitales sv = (SignosVitales) t[i];
            if (sv.alarma()) cant++;
        }
        return cant;
    }
}
    
```

- Podemos asegurar que **t[i]** mantiene una referencia a un objeto de clase **SignosVitales**.

27

GENERICIDAD

- La clase **Coleccion** es **genérica**, permite factorizar el comportamiento compartido definiendo un patrón general.
- Las clases **Correo**, **Inventario** y **RegistroSignosVitales** extienden a **Coleccion** con servicios específicos.
- Estamos modelando genericidad usando herencia.
- En las materias que siguen se presenta un mecanismo para definir clases genéricas a partir de **polimorfismo paramétrico**.
- Java brinda recursos limitados para soportar este concepto.

```

01:00
02:01
03:11
04:10
05:10
06:10
07:11
08:11
09:10
10:10
11:01
12:11
13:11
14:10
15:10
16:11
17:11
18:11
19:11
20:11
21:11
22:11
23:11
24:11
25:11
26:11
27:11
28:11
29:11
30:11
31:11
32:11
33:11
34:11
35:11
36:11
37:11
38:11
39:11
40:11
41:11
42:11
43:11
44:11
45:11
46:11
47:11
48:11
49:11
50:11
51:11
52:11
53:11
54:11
55:11
56:11
57:11
58:11
59:11
60:11
61:11
62:11
63:11
64:11
65:11
66:11
67:11
68:11
69:11
70:11
71:11
72:11
73:11
74:11
75:11
76:11
77:11
78:11
79:11
80:11
81:11
82:11
83:11
84:11
85:11
86:11
87:11
88:11
89:11
90:11
91:11
92:11
93:11
94:11
95:11
96:11
97:11
98:11
99:11
100:11
    
```

28